

NAME : MADAN REDDY A

REG NO: 24BECS129

1.BREADTH FIRST SEARCH

```
from queue import PriorityQueue

def bfs(graph, start, target, heuristics):
    visited = set()
    pq = PriorityQueue()
    pq.put((heuristics[start], start, [start]))
    visited.add(start)

    while not pq.empty():
        h, node, path = pq.get()

        if node == target:
            return path, h

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                pq.put((heuristics[neighbor], neighbor, path + [neighbor]))

    return None, None

if __name__ == "__main__":
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': [],
        'F': []
    }
```

```
heuristics = {
    'A': 10,
    'B': 8,
    'C': 5,
    'D': 6,
    'E': 4,
    'F': 0
}
```

```
path, cost = bfs(graph, 'A', 'F', heuristics)
```

```
print(f"NAME : MADAN REDDY A")
```

```
print(f"ROLL NO : 24BECS129")
```

```
print(f"Path found by BFS: {path} with Target Heuristic: {cost}")
```

OUT PUT:

```
PS C:\aiml> & C:\Python314\python.exe c:/aiml/BESTfirstSearch.py  
NAME : MADAN REDDY A  
ROLL NO : 24BECS129  
Path found by BFS: ['A', 'C', 'F'] with Target Heuristic: 0  
PS C:\aiml> █
```

2. BFS AND DFS

```

from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []

    while queue:
        vertex = queue.popleft()
        order.append(vertex)
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []
    visited.add(start)
    order.append(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
    return order

#representing the graph as an adjacency list
graph = {
    'A': ['B', 'C'],

```

```

def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []
    visited.add(start)
    order.append(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
    return order

#representing the graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
print(f"NAME : MADAN REDDY A")
print(f"ROLL NO : 24BECS129")
print("BFS Traversal starting from 'A';", bfs(graph, 'A'))
print("DFS Traversal starting from 'A';", dfs(graph, 'A'))

```

OUT PUT :

```

PS C:\aiml> & C:\Python314\python.exe c:/aiml/bfsanddfs
NAME : MADAN REDDY A
ROLL NO : 24BECS129
BFS Traversal starting from 'A'; ['A', 'B', 'C', 'D', 'E', 'F']
DFS Traversal starting from 'A'; ['A', 'B', 'D', 'E', 'F', 'C']
PS C:\aiml> 

```

3.MEAN END ANALYSIS :

```
class MeansEndAnalysis:
    def __init__(self, operators):
        self.operators = operators

    def solve(self, current, goal):
        print(f"Current state: {current} | Goal state: {goal}")

        # Goal achieved
        if current == goal:
            return []

        # Find the first difference
        diff = self.find_difference(current, goal)

        if not diff:
            return []

        # Select an operator to resolve the difference
        op = self.select_operator(diff)

        if not op:
            print(f"No operator found to resolve difference: {diff}")
            return None

        # Solve preconditions first
        preconditions_path = self.solve(current, op['precond'])

        if preconditions_path is None:
            return None
```

```
# Apply the operator
new_state = current.copy()
new_state.update(op['effect'])

# Solve the remaining goal
remaining_path = self.solve(new_state, goal)

if remaining_path is None:
    return None

return preconditions_path + [op['name']] + remaining_path

def find_difference(self, current, goal):
    for key in goal:
        if current.get(key) != goal[key]:
            return (key, goal[key])
    return None

def select_operator(self, diff):
    key, val = diff
    for op in self.operators:
        if op['effect'].get(key) == val:
            return op
    return None
```



```
if __name__ == "__main__":
    operators = [
        {
            'name': 'Drive_car',
            'precond': {
                'has_car': True,
                'at_home': True
            },
            'effect': {
                'at_work': True,
                'at_home': False
            }
        },
        {
            'name': 'Buy_car',
            'precond': {
                'has_money': True,
                'has_car': False
            },
            'effect': {
                'has_car': True
            }
        }
    ]
```

```

# Initial state
current_state = {
    'has_money': True,
    'has_car': False,
    'at_home': True,
    'at_work': False
}

# Goal state
goal_state = {
    'at_work': True
}

print(f"NAME : MADAN REDDY A")
print(f"ROLL NO : 24BECS129")

mea = MeansEndAnalysis(operators)

plan = mea.solve(current_state, goal_state)

if plan:
    print("\nPlan found:")
    for step in plan:
        print(step)
else:
    print("\nNo solution found.")

```

OUTPUT:

```

NAME : MADAN REDDY A
ROLL NO : 24BECS129
Current state: {'has_money': True, 'has_car': False, 'at_home': True, 'at_work': False} | Goal state: {'at_work': True}
Current state: {'has_money': True, 'has_car': False, 'at_home': True, 'at_work': False} | Goal state: {'has_car': True, 'at_home': True}
Current state: {'has_money': True, 'has_car': False, 'at_home': True, 'at_work': False} | Goal state: {'has_money': True, 'has_car': False}
Current state: {'has_money': True, 'has_car': True, 'at_home': True, 'at_work': False} | Goal state: {'has_car': True, 'at_home': True}
Current state: {'has_money': True, 'has_car': False, 'at_home': False, 'at_work': True} | Goal state: {'at_work': True}

Plan found:
Buy_car
Drive_car
PS C:\aiml>

```